

Hybrid Convolutional Transformer for Face-Image Forensic Hashing

Architecture and top-level code walkthrough

Convolutional_Transformer repository

June 1, 2026

Scope of this walkthrough

- Covers every top-level source file in the repository root.
- Python: `CNN.py`, `tokenizer.py`, `transformer.py`, `augmentations.py`, `loss.py`, `train.py`, `eval.py`.
- HPC launch scripts: `submit_CvT_train.sh`, `submit_4gpu.sh`.
- Sub-folders are intentionally excluded.
- Focus: exact tensor shapes, kernel sizes, layer counts, optimization behavior, distributed training, evaluation, and implementation caveats.

Problem formulation

Goal

Learn a semantic forensic hash that is stable under benign global image degradation but changes when a localized manipulation is introduced.

Anchor

Pristine face image

Positive view

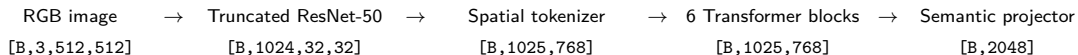
Global benign blur, noise, and brightness shift

Hard negative

Localized heavily blurred rectangular region

Every view is encoded into a normalized 2,048-dimensional vector. Training pulls the anchor toward the benign view and pushes it away from the forged view and unrelated images.

End-to-end architecture



- The CNN preserves local forensic texture information on a 32×32 grid.
- The tokenizer turns each grid cell into one token and prepends a learnable classification token.
- Self-attention allows every spatial location to interact with every other location.
- The final classification token becomes the compact semantic hash.

Parameter budget

Module	Trainable parameters
Truncated ResNet-50 backbone	8,543,296
Spatial tokenizer	1,575,168
Six transformer blocks	42,527,232
Semantic projector	1,579,008
Total	54,224,704

- One transformer block has 7,087,872 trainable parameters.
- The six-block transformer head contributes about 78.4% of the total.
- Augmentation kernels are registered buffers, not trainable parameters.

CNN.py: truncated ResNet-50

- Implements ResNet bottleneck blocks from scratch.
- Uses the standard ResNet-50 stage depths [3, 4, 6, 3], but omits the last stage: only [3, 4, 6] are constructed.
- Also omits adaptive average pooling and the classification layer.
- Input is resized to 512×512 before it reaches the backbone.
- Output is a spatial map, not a class score: $[B, 1024, 32, 32]$.

Why truncate?

Keeping the 32×32 map supplies 1,024 spatial tokens. A complete ResNet-50 would downsample further and discard useful local anomaly detail.

CNN stem: exact downsampling

Operation	Kernel	Stride	Padding	Output shape
Input	–	–	–	[B, 3, 512, 512]
Conv2d	7×7	2	3	[B, 64, 256, 256]
BatchNorm2d + ReLU	–	–	–	[B, 64, 256, 256]
MaxPool2d	3×3	2	1	[B, 64, 128, 128]

For convolution or pooling,

the spatial output is

$$\left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1.$$

All convolutions use `bias=False` when immediately followed by batch normalization.

Residual bottleneck block

Each Bottleneck applies:

$$x \rightarrow \underbrace{\text{Conv}_{1 \times 1}}_{\text{reduce}} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \underbrace{\text{Conv}_{3 \times 3}}_{\text{spatial}} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \underbrace{\text{Conv}_{1 \times 1}}_{\text{expand by 4}} \rightarrow \text{BN}.$$

Then:

$$y = \text{ReLU}(F(x) + D(x)).$$

- Expansion factor: 4.
- The 3×3 convolution uses padding 1 and carries the stage stride.
- $D(x)$ is either the identity or a 1×1 projection plus BatchNorm.
- ReLU is configured with `inplace=False`.

CNN stages: dimensions and block counts

Stage	Blocks	Internal channels	Expanded output	First-block stride	Grid	Parameters	
Stem	–	–	64	2 then pool 2	128×128	9,536	A projection shortcut
Layer 1	3	64	256	1	128×128	215,808	
Layer 2	4	128	512	2	64×64	1,219,584	
Layer 3	6	256	1,024	2	32×32	7,098,368	
Output	–	–	1,024	–	32×32	8,543,296 total	

is created whenever stride changes or channel counts do not match the expanded output. CNN convolutions use Kaiming-normal initialization; BatchNorm scales start at 1 and biases at 0.

tokenizer.py: spatial-to-sequence bridge

The tokenizer preserves all $32 \times 32 = 1,024$ CNN grid positions:

$$[B, 1024, 32, 32] \rightarrow [B, 1024, 1024] \rightarrow [B, 1024, 768].$$

- 1 view flattens the spatial axes.
- 2 `permute(0,2,1)` changes channel-first CNN layout into token-first layout.
- 3 A learned linear projection maps each 1,024-channel CNN vector to 768 dimensions.
- 4 A learnable $[1, 1, 768]$ classification token is prepended.
- 5 A learnable $[1, 1025, 768]$ absolute positional embedding is added.

Tokenizer parameters and initialization

Object	Shape	Parameters
Patch projection weight	[768, 1024]	786,432
Patch projection bias	[768]	768
Classification token	[1, 1, 768]	768
Positional embedding	[1, 1025, 768]	787,200
Total	–	1,575,168

- Classification token and positional embedding: truncated normal, standard deviation 0.02.
- Projection weight: Xavier-uniform.
- Projection bias: zeros.
- There is no patchifying convolution: tokens correspond directly to CNN grid cells.

transformer.py: six pre-norm blocks

The model creates six identical CustomTransformerBlock instances:

$$Z'_\ell = Z_\ell + \text{MHSA}(\text{LN}(Z_\ell)),$$

$$Z_{\ell+1} = Z'_\ell + \text{MLP}(\text{LN}(Z'_\ell)).$$

- Hidden dimension: 768.
- Sequence length: 1,025.
- Attention heads: 12.
- Per-head dimension: $768/12 = 64$.
- MLP ratio: 4, giving a 3,072-dimensional hidden layer.
- Both sublayers have residual additions.

Multi-head self-attention in detail

- 1 One fused linear layer projects $[B, 1025, 768]$ to $[B, 1025, 2304]$.
- 2 `chunk(3, dim=-1)` produces query, key, and value tensors.
- 3 Each tensor is reshaped to $[B, 12, 1025, 64]$.
- 4 PyTorch `scaled_dot_product_attention` computes full, bidirectional self-attention with no mask and no dropout.
- 5 Heads are concatenated back to $[B, 1025, 768]$.
- 6 A final linear layer mixes the concatenated heads.

The mathematical operation per head is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{64}}\right) V.$$

Attention implementation details

- `attn_mask=None`: all tokens can attend to all tokens.
- `is_causal=False`: this is an encoder, not an autoregressive decoder.
- `dropout_p=0.0`: attention is deterministic even in training.
- `scale=None`: PyTorch uses the default $1/\sqrt{d_k}$ factor.
- On a compatible CUDA stack and GPU, PyTorch may dispatch the SDPA call to an optimized Flash Attention backend.
- The code requests SDPA; the actual selected backend depends on runtime hardware, data type, and PyTorch configuration.

MLP and optional checkpointing

The second transformer sublayer is:

$$768 \xrightarrow{\text{Linear}} 3072 \xrightarrow{\text{GELU}} 3072 \xrightarrow{\text{Linear}} 768.$$

- Each block exposes `use_checkpoint=False` by default.
- Training can set it to true for every block via `-enable-checkpointing`.
- During training, `checkpoint(..., use_reentrant=False)` recomputes block activations during backward to reduce activation memory.
- The trade-off is extra computation during backpropagation.

One transformer block: parameter accounting

Component	Parameters
Two LayerNorm layers	3,072
Fused QKV projection: $768 \rightarrow 2304$	1,771,776
Attention output projection: $768 \rightarrow 768$	590,592
MLP first linear: $768 \rightarrow 3072$	2,362,368
MLP second linear: $3072 \rightarrow 768$	2,360,064
One block	7,087,872
Six blocks	42,527,232

After the sixth transformer block, only token index 0 is retained:

$$[B, 1025, 768] \xrightarrow{\text{select CLS}} [B, 768] \xrightarrow{\text{Linear}} [B, 2048] \xrightarrow{\text{GELU+LayerNorm}} [B, 2048].$$

- Linear layer: $768 \rightarrow 2,048$.
- Projector parameters: 1,579,008.
- The loss later L2-normalizes hashes onto the unit hypersphere.
- The model itself returns the unnormalized 2,048-D vector.

augmentations.py: fixed depthwise filter bank

Raw filters are non-trainable registered buffers. Each RGB channel is filtered independently with

	Filter	Kernel	Default parameters	Purpose	
groups=3.	RawGaussianFilter	$K \times K$	$K = 7, \sigma = 1.5$	Gaussian blur	For odd
	RawBoxFilter	$K \times K$	$K = 5$	Uniform averaging	
	RawMotionFilter	$K \times K$	$K = 7$	Diagonal camera shake	

K , padding is $K//2$, preserving image height and width. Each kernel is reshaped to $[1, 1, K, K]$ and repeated to $[3, 1, K, K]$.

Gaussian, box, and motion kernels

Gaussian kernel

Coordinates run from $-P$ to P , where $P = K//2$. The 1-D Gaussian is $g(x) = \exp(-x^2/(2\sigma^2))$. The 2-D kernel is the normalized outer product $gg^\top$.

Box kernel

Every one of the K^2 entries is $1/K^2$, so brightness is preserved.

Motion kernel

A normalized $K \times K$ identity matrix places $1/K$ along the diagonal, producing an anisotropic diagonal smear.

Global benign augmentation

For an entire batch, `GlobalBenignAugmentation` randomly selects one blur:

- Gaussian: 5×5 , $\sigma = 1.0$.
- Gaussian: 7×7 , $\sigma = 2.0$.
- Box: 5×5 .
- Motion: 7×7 .

It then adds pixelwise Gaussian noise with standard deviation sampled uniformly from $[0.01, 0.05]$, multiplies by a brightness scalar sampled uniformly from $[0.9, 1.1]$, and clamps pixels to $[0, 1]$.

Implementation detail

The bank is an `nn.ModuleList`; therefore `.to(device)` moves every registered filter buffer to the GPU.

Localized malicious forgery

Training constructs `LocalMaliciousForgery(tamper_size=(96,192))`.

- ① Pick one heavy Gaussian blur for the batch: 31×31 kernel with $\sigma \in \{5, 6, 7, 8\}$.
- ② For each image, independently sample rectangle height and width from $[96, 192]$ pixels.
- ③ Sample a rectangle origin and set the binary mask region to 1.
- ④ Blur the full image, then use the mask to blend blurred pixels only into the selected region.

The class default is $(64, 128)$, but `train.py` overrides it.

Soft mask for forgery blending

A binary rectangle would create an obvious hard edge. The code feathers that edge by convolving the one-channel mask with a Gaussian:

$$M_{\text{soft}} = M_{\text{hard}} * G_{31 \times 31, \sigma=7}.$$

The final image is:

$$x_{\text{tampered}} = M_{\text{soft}} \odot x_{\text{blurred}} + (1 - M_{\text{soft}}) \odot x.$$

- Soft-mask kernel: 31×31 , $\sigma = 7$, padding 15.
- One randomized box is made per image.
- Heavy-blur choice is shared by the batch.
- CUDA scalar indices are converted to Python integers with `.item()`.

loss.py: asymmetric contrastive objective

Let h_o , h_g , and h_t denote pristine, globally augmented, and locally tampered hashes. First normalize:

$$z = \frac{h}{\|h\|_2}.$$

With temperature $\tau = 0.05$ and hard-negative weight $\alpha = 20$:

$$s_i^+ = \frac{z_{o,i}^\top z_{g,i}}{\tau}, \quad s_i^{\text{hard}} = \frac{z_{o,i}^\top z_{t,i}}{\tau}.$$

Anchor direction is asymmetric: pristine images are anchors, benign images are positives, and tampered images are weighted hard negatives.

Implemented denominator

For each anchor, compare against all gathered benign views and remove its matched positive from that set. The implemented loss is:

$$\mathcal{L}_i = - \left[s_i^+ - \log \left(\sum_{j \neq i} \exp(s_{ij}) + \alpha \exp(s_i^{\text{hard}}) \right) \right].$$

- $s_{ij} = \mathbf{z}_{o,i}^\top \mathbf{z}_{g,j} / \tau$.
- Positive entries are replaced with $-\infty$ before exponentiation.
- The batch loss is the mean of per-sample losses.
- Similarity computation is cosine similarity because vectors are normalized.

Distributed negative gathering

In multi-GPU mode:

- Each rank gathers benign-view embeddings from all ranks.
- Local batch size B becomes global context size $N_{\text{global}} = B \times \text{WORLD_SIZE}$.
- `dist.all_gather` does not preserve autograd for gathered tensors.
- The code replaces the gathered local shard with its original tensor, preserving gradients for that local shard.
- The local positive columns begin at `rank * B`; these positions are masked out of the denominator.

This expands the negative pool without sending gradients through remote shards.

Verify the intended objective

The implementation excludes the positive term from the denominator. Standard InfoNCE / NT-Xent commonly includes the positive alongside negatives in the denominator. Excluding it can produce negative loss values when the positive similarity dominates.

- The slides describe the repository exactly as implemented.
- The code comments explicitly state that exclusion is intentional.
- Before comparing results against standard NT-Xent baselines, confirm whether the desired objective is standard InfoNCE or this custom ratio.

train.py: dataset and data loading

PristineFacesDataset:

- Recursively discovers .jpg, .jpeg, and .png files.
- Converts each image to RGB.
- Resizes every image to 512×512 .
- Converts pixels to a float tensor in $[0, 1]$.
- Leaves augmentation to the GPU-side training loop.

DataLoader:

- Uses pinned memory, `drop_last=True`, and prefetch factor 4.
- Persistent workers are enabled when worker count is positive.
- Uses `DistributedSampler` under DDP; otherwise uses shuffle.

Training batch construction

For each pristine mini-batch x :

$$x_{\text{global}} = A_{\text{benign}}(x), \quad x_{\text{tampered}} = A_{\text{local}}(x).$$

Augmentations run inside `torch.no_grad()`, then:

$$x_{\text{all}} = \text{cat}(x, x_{\text{global}}, x_{\text{tampered}}) \in \mathbb{R}^{3B \times 3 \times 512 \times 512}.$$

A single model call returns `[3B, 2048]`, then `chunk(3)` recovers anchor, positive, and hard-negative hashes. This avoids three separate forward calls.

Optimization configuration

Setting	Default
Optimizer	AdamW
Base learning rate	10^{-4}
Distributed learning rate	base LR $\times$ world size
Weight decay	10^{-4}
Epochs	100 from Python CLI
Warmup	10 epochs
Gradient clipping	max norm 1.0
Temperature τ	0.05
Hard-negative weight α	20.0
Checkpoint interval	10 epochs

Learning-rate schedule

`build_lr_lambda` applies linear warmup followed by cosine decay:

$$\lambda(e) = \begin{cases} \frac{e + 1}{E_{\text{warmup}}}, & e < E_{\text{warmup}}, \\ \frac{1}{2} \left(1 + \cos \left(\pi \frac{e - E_{\text{warmup}}}{E_{\text{total}} - E_{\text{warmup}}} \right) \right), & \text{otherwise.} \end{cases}$$

- Scheduler step occurs after each epoch.
- Learning rate is scaled linearly by the number of processes.
- A denominator guard of `max(1, ...)` handles edge cases.

CUDA, AMP, and DDP setup

- Training requires CUDA; there is no CPU training fallback.
- If `LOCAL_RANK`, `RANK`, and `WORLD_SIZE` exist, NCCL is initialized. Partial distributed environments are rejected.
- Without those variables, the code uses one GPU for easier debugging.
- TF32 matmul and cuDNN TF32 are enabled.
- AMP selects bfloat16 when supported, otherwise float16.
- GradScaler is enabled only for float16.
- DDP uses one device per process and `find_unused_parameters=False`.

Checkpointing and resume behavior

After every epoch on rank 0, `train.py` saves:

- zero-based epoch index,
- unwrapped model state,
- optimizer state,
- scheduler state,
- averaged epoch loss.

The rolling path defaults to `latest_checkpoint.pth`. Every tenth epoch, a named file such as `hybrid_encoder_epoch_010.pth` is also written. If the rolling checkpoint already exists, training automatically resumes at the following epoch.

eval.py: evaluation workflow

eval.py is an executable evaluation script with fixed constants:

- checkpoint: hybrid_encoder_epoch_150.pth,
- evaluation root: a hard-coded cluster path,
- device: CUDA when available, otherwise CPU,
- random seed: 42.

It loads the model, computes normalized hashes for `real`, `positive`, and `negative` directories, and stores results in a directory named from the checkpoint epoch.

Evaluation matrices and threshold search

If R , P , and N contain normalized embedding rows, the script computes:

$$M_{RP} = RP^{\top}, \quad M_{RN} = RN^{\top}, \quad M_{PN} = PN^{\top}.$$

- Diagonal entries are treated as matched image pairs.
- Off-diagonal entries are treated as unmatched pairs.
- Threshold candidates are sampled from $[-1, 1]$ in increments of 0.001.
- A score above threshold predicts a benign positive pair.
- The threshold maximizing accuracy across matched RP and RN diagonals is saved.

Evaluation artifacts

The output directory contains:

- three serialized embedding tensors,
- `similarity_matrices.npz`,
- three 300-DPI heatmaps,
- `statistics.txt`.

The statistics report includes diagonal and off-diagonal mean, standard deviation, minimum, maximum, five random examples, the diagonal-off-diagonal gap, hard/easy real-negative cases, and best threshold accuracy.

submit_CvT_train.sh: eight-GPU job

SLURM resource	Requested value
Nodes / tasks	1 / 1
GPUs	8
CPUs	64
Memory	256 GB
Wall time	48 hours
Partition / QoS	dgx / dgx

The script supports micromamba, mamba, or conda activation,

launches `torchrun -nproc_per_node=8`, enables checkpointing, defaults to 48 images per GPU and 500 epochs, and forwards additional command-line arguments.

submit_4gpu.sh: four-GPU job

SLURM resource	Requested value
Nodes / tasks	1 / 1
GPUs	4
CPUs	32
Memory	128 GB
Wall time	24 hours
Partition / QoS	dgx / dgx

This variant requires mamba, launches four ranks, enables

checkpointing, and uses the same default 48 images per GPU and 500 epochs. Both scripts print environment details, invoke `nvidia-smi`, capture the exit code, and preserve strict shell error handling outside the training command.

HPC environment variables

Both submission scripts configure:

- `OMP_NUM_THREADS=8,`
- `NCCL_DEBUG=INFO,`
- `NCCL_IB_DISABLE=0,`
- `NCCL_P2P_DISABLE=0,`
- `PYTHONUNBUFFERED=1.`

User-overridable values include conda environment, project directory, dataset path, batch size, worker count, rolling checkpoint path, and epoch count.

Effective batches

Script defaults yield 192 images per step on four GPUs and 384 images per step on eight GPUs, before the training loop triples each batch into views.

Implementation notes to verify

- `train.py` defaults to dataset path `Faces/train`; both shell scripts default to `Faces`. Recursive discovery may make either valid, but the intended layout should be documented.
- The Python CLI defaults to 100 epochs while both shell scripts override this with 500 epochs.
- `eval.py` accepts only top-level `*.jpg` files in each evaluation category; it ignores JPEG variants, PNG files, and nested files.
- `eval.py` has hard-coded checkpoint and dataset paths rather than CLI arguments.
- The contrastive denominator differs from standard InfoNCE / NT-Xent; confirm the intended research formulation.

Evaluation assumptions to verify

- Similarity diagonals assume category folders use matching sorted filenames and compatible lengths.
- `compute_stats` constructs a square identity mask from row count; non-square similarity matrices would fail or be interpreted incorrectly.
- Five random matched pairs require at least five samples.
- Off-diagonal sampling requires at least two samples.
- Empty image directories cause `torch.stack` to fail.
- Evaluation uses `torch.load` without `weights_only=True`; only trusted checkpoints should be loaded.

Training signal summary

Pristine anchor	Benign positive	Local hard negative
Original resized RGB image	Global blur + noise + brightness shift	Feathered local heavy blur
Should remain close to positive	Models' acceptable acquisition changes	Should move away from anchor

Core idea

CNN features preserve fine spatial evidence; transformer attention integrates evidence globally; contrastive training makes the final hash robust to benign degradation and sensitive to localized manipulation.

Complete tensor-shape trace

Stage	Operation	Shape
Input	RGB batch	[B,3,512,512]
CNN stem	7×7 conv, 3×3 max pool	[B,64,128,128]
CNN layer 1	3 bottlenecks	[B,256,128,128]
CNN layer 2	4 bottlenecks	[B,512,64,64]
CNN layer 3	6 bottlenecks	[B,1024,32,32]
Tokenizer	flatten grid	[B,1024,1024]
Tokenizer	project channels	[B,1024,768]
Tokenizer	prepend CLS, add positions	[B,1025,768]
Transformer	6 blocks	[B,1025,768]
Projector	select CLS	[B,768]
Projector	linear, GELU, LayerNorm	[B,2048]
Loss	L2 normalize	[B,2048]

Top-level file map

File	Responsibility
CNN.py	Truncated ResNet-50 spatial backbone
tokenizer.py	CNN grid to transformer-token sequence
transformer.py	Attention blocks and semantic projector
augmentations.py	Benign and malicious GPU-side image transforms
loss.py	Asymmetric contrastive loss and distributed gather
train.py	Dataset, model assembly, optimization, DDP, checkpoints
eval.py	Embeddings, similarity matrices, thresholding, plots
submit_CvT_train.sh	Eight-GPU SLURM launcher
submit_4gpu.sh	Four-GPU SLURM launcher

Questions?